

Давайте пройдемося покроково через приклад прямого проходу моделі Word2Vec, використовуючи речення "The cat sat on the mat." Ми використаємо спрощену версію моделі [Skip-Gram](#) для цього прикладу.

Крок 1: Попередня обробка

1. Токенізація: Розділимо речення на слова.

- Токени: ["The", "cat", "sat", "on", "the", "mat"]

2. Видалення дублікатів: Визначимо унікальні слова у словнику.

- Словник/Корпус: {"the", "cat", "sat", "on", "mat"}

3. Індексція словника: Присвоїмо індекс кожному слову.

- the: 0, cat: 1, sat: 2, on: 3, mat: 4

Крок 2: Створення векторів слів

4. One-hot encoding: Для словника розміру 5 здійснимо кодування для кожного слова.

- the: [1, 0, 0, 0, 0]

- cat: [0, 1, 0, 0, 0]

- sat: [0, 0, 1, 0, 0]

- on: [0, 0, 0, 1, 0]

- mat: [0, 0, 0, 0, 1]

5. Початкові вектори для слів: ініціалізуємо випадкові вектори для кожного слова. Припустимо, ми хочемо 3-вимірні вектори слів.

- **W1:**

[[0.1, 0.2, 0.3], # the
[0.3, 0.4, 0.2], # cat
[0.5, 0.6, 0.1], # sat
[0.7, 0.8, 0.4], # on
[0.9, 0.3, 0.1]] # mat

Крок 3: Визначення контексту

6. Вікно контексту: Визначимо контекст для цільового слова. Для цільового слова "cat" на індексі 1 візьмемо одне слово з кожної сторони.

- Слова контексту для "cat": `["the", "sat"]` (індекси: 0, 2)

Крок 4: Forward pass

7. Вхідний вектор: Використовуємо one-hot encoding для цільового слова "cat".

- Вхідний вектор **I**: [0, 1, 0, 0, 0]

8. Подання на прихованому шарі: помножимо вхідний вектор на вагову матрицю **W1**:

$$\mathbf{h1} = \mathbf{I} \cdot \mathbf{W1}$$

- Обчислення:

$$\begin{aligned} [0, 1, 0, 0, 0] \times & \begin{bmatrix} 0.1, 0.2, 0.3, \\ 0.3, 0.4, 0.2, \\ 0.5, 0.6, 0.1, \\ 0.7, 0.8, 0.4, \\ 0.9, 0.3, 0.1 \end{bmatrix} \end{aligned}$$

В результаті отримаємо деякий вектор $\mathbf{h1} = [0.3, 0.4, 0.2]$.

Крок 5: Обчислення виходу

9. Вихідні ваги: Ініціалізуємо ще одну вагову матрицю $\mathbf{W2}$ для вихідного шару

- Припустимо:

...

$\mathbf{W2}$:

[[0.1, 0.2, 0.3, 0.4, 0.5], # від першої компоненти $h1$

[0.3, 0.4, 0.5, 0.6, 0.7], # від другої компоненти $h1$

[0.5, 0.6, 0.7, 0.8, 0.9]] # від третьої компоненти $h1$

10. Обчислення виходу: помножимо вихід прихованого шару на ваги виходу.

$\mathbf{output} = \mathbf{h.W2} = [0.25, 0.34, 0.43, 0.52, 0.61]$ (розмірність $\mathbf{output}$: 5)

- Це дасть нам вектор оцінок для кожного слова в словнику.

Крок 6: Softmax для отримання ймовірностей

11. Функція [softmax](#): Перетворюємо вихідні оцінки на ймовірності.

$$P(y|x) = \frac{e^{\text{score}_i}}{\sum_j e^{\text{score}_j}}$$

Це дасть нам ймовірності для кожного слова в словнику, що є цільовим словом, з урахуванням контексту.

Давайте порахуємо softmax для output = [0.25, 0.34, 0.43, 0.52, 0.61]

Формула softmax: $P(y|x) = \exp(\text{score}_i) / \sum \exp(\text{score}_j)$

1. Спочатку рахуємо exp для кожного значення:

$$\exp(0.25) = 1.2840$$

$$\exp(0.34) = 1.4049$$

$$\exp(0.43) = 1.5373$$

$$\exp(0.52) = 1.6821$$

$$\exp(0.61) = 1.8404$$

2. Рахуємо суму всіх exp:

$$\sum \exp(\text{score}_j) = 1.2840 + 1.4049 + 1.5373 + 1.6821 + 1.8404 = 7.7487$$

3. Ділимо кожне exp на суму:

$$P(\text{"the"}) = 1.2840 / 7.7487 = 0.1657 \text{ (16.57\%)}$$

$$P(\text{"cat"}) = 1.4049 / 7.7487 = 0.1814 \text{ (18.14\%)}$$

$$P(\text{"sat"}) = 1.5373 / 7.7487 = 0.1984 \text{ (19.84\%)}$$

$$P(\text{"on"}) = 1.6821 / 7.7487 = 0.2171 \text{ (21.71\%)}$$

$$P(\text{"mat"}) = 1.8404 / 7.7487 = 0.2374 \text{ (23.74\%)}$$

Перевірка: сума всіх ймовірностей = 1 (100%)

Це фінальні ймовірності для кожного слова в словнику бути контекстним словом для цільового слова "cat". Модель прогнозує, що найбільш

ймовірним контекстним словом є "mat" (23.74%), а найменш імовірним - "the" (16.57%).

Давайте знайдемо loss для цього прикладу. У нас є два контекстних слова для "cat": "the" та "sat".

1. Для слова "the" (індекс 0):

- True distribution: [1, 0, 0, 0, 0]

- Predicted probability: 0.1657

2. Для слова "sat" (індекс 2):

- True distribution: [0, 0, 1, 0, 0]

- Predicted probability: 0.1984

Використовуємо крос-ентропію для кожного слова:

$Loss = -\sum(true_label * \log(predicted_probability))$

1. Loss для "the":

$-\log(0.1657) = 1.7977$

2. Loss для "sat":

$-\log(0.1984) = 1.6178$

3. Загальний loss - це середнє значення:

$Total\ Loss = (1.7977 + 1.6178) / 2 = 1.7078$

Інтерпретація:

- Loss близький до 1.71 показує, що модель досить далека від ідеальних передбачень
- Ідеальний випадок був би, якби модель давала ймовірності близькі до 1.0 для правильних контекстних слів
- Чим менший loss, тим краще модель передбачає контекст

Для оптимізації цього loss'у потрібно:

1. Обчислити градієнти по відношенню до $W1$ та $W2$
2. Оновити ваги використовуючи градієнтний спуск
3. Повторювати процес для всіх слів у корпусі

У моделі Word2Vec embedding слова можна отримати двома способами (тут і далі значення матриць ініціалізовані випадково на самому початку; насправді, їх потрібно брати після навчання моделі):

1. Напрямку з матриці $W1$:

- Якщо нам потрібен embedding для слова "cat" (індекс 1), ми просто беремо другий рядок з матриці $W1$:

`embedding_cat = W1[1] # [0.3, 0.4, 0.2]`

2. Через прихований шар ($h1$):

- Беремо one-hot вектор для слова (наприклад, для "cat": $[0, 1, 0, 0, 0]$)
- Множимо його на $W1$
- Отриманий вектор $h1$ і буде embedding'ом слова

Обидва способи дадуть однаковий результат, тому що множення one-hot вектора на матрицю $W1$ фактично просто обирає відповідний рядок з $W1$.

Важливо розуміти:

1. Матриця $W1$ - це і є матриця embeddings всіх слів
2. Кожен рядок $W1$ - це embedding відповідного слова
3. Розмірність embedding'a - це кількість стовпців $W1$ (в нашому прикладі 3)
4. Після тренування моделі ці embedding'и будуть відображати семантичну близькість слів у просторі меншої розмірності

В нашому прикладі:

```
embeddings = {  
    "the": [0.1, 0.2, 0.3],  
    "cat": [0.3, 0.4, 0.2],  
    "sat": [0.5, 0.6, 0.1],  
    "on": [0.7, 0.8, 0.4],  
    "mat": [0.9, 0.3, 0.1]  
}
```